- K-d trees are great with low- and medium-dimensional spaces, but suffer sparsity of high-dimensional spaces.
- K-d trees work better on static datasets, because we can build balanced trees on construction, but `insert` and `remove` are not self-balancing operations.

---

[1] Similarity Search trees.

[2] "Multidimensional binary search trees used for associative searching". Communications of the ACM.

[3] Here we choose points arbitrarily, to obtain a clearer visualization. In section 9.3.3 we will explain how to make this choice programmatically to obtain a balanced tree.

[4] **D**on't **R**epeat **Y**ourself: in this case we have some code duplication, that makes the code slightly less maintainable.

[5] Immutability of data structures is a key point of functional programming. It has several advantages, from being intrinsically thread-safe to easier debug.

[6] https://en.wikipedia.org/wiki/Master_theorem_(analysis_of_algorithms)

[7] `n*log(n)` is often referred to as linearitimic, as a *crasis* of linear and logarithmic.

[8] Note that we could also use `N`'s predecessor, in a symmetrical way.

[9] This is implied by invariant 4 as described in section 9.2.2

[10] This method is usually denoted as k-nearest-neighbor search, but IMHO the use of `k` here could cause confusion with the dimension of the tree, hence we will just use `m` or `n` to indicate the number of points we look for.

[11] As mentioned, this is also referred to as k-nearest-neighbor in literature.

[12] Since we can define an ad-hoc unit measure for the area, it is always possible to imagine so.

[13] The one leaf that is closest to the target point, as shown in listing 9.9.

[14] In a perfect world…

[15] The simplex algorithm is an ingenious optimization method. It is not related nor helped by k-d trees and, as such, out of scope for this chapter, but it's an interesting reading, and you can read more for example here: https://en.wikipedia.org/wiki/Simplex_algorithm